

SOFTWARE ENGINEERING LAB 3

Name Sanika Patil

SRN PES1U24AM248

Section E

Scenario Webhook Ingestion & Retry Mechanism Hub

1. Scenario Review

The Webhook Ingestion & Retry Mechanism Hub is designed to receive webhook events from external systems, verify and store them, deliver them to target endpoints, and recover from delivery failures. The system also provides delivery-status information so that an operator can identify failures and take action when needed.

Functional requirements

- Receive incoming webhook requests from external integration systems.
- Verify the incoming webhook and store the accepted event.
- Deliver the stored webhook payload to the intended target endpoint.
- Automatically retry deliveries that fail.
- Allow manual retry of a webhook that has reached the dead-letter state.
- Provide delivery status for monitoring and inspection.

Main design concerns

Reliability: Failed deliveries must be recoverable instead of being silently lost.

Security: Incoming requests should be verified before they enter the processing flow. **Performance:** Webhook processing should remain responsive as event volume increases. **Monitoring:** Operators need clear information about successful, failed, retried, and dead-lettered events.

2. Architectural Style Analysis

Layered Architecture

Advantages

Clear separation of presentation, business, and data responsibilities. It is easy to understand and maintain.

Limitations

The layers can become dependent on one another, and scaling one part independently is less convenient.

Fit for this system

Moderate fit

Architecture selection

Microservices Architecture

Advantages

Small independent services support fault isolation, independent scaling, and separate deployment.

Limitations

It introduces more operational work, network communication, and possible

data-consistency issues.

Fit for this system

Best fit

Client-Server Architecture

Advantages

Centralised control and simple deployment make it straightforward for smaller applications.

Limitations

A central server can become a bottleneck and a single point of failure.

Fit for this system

Lower fit

Microservices Architecture is the most suitable choice because the webhook system naturally separates into independent responsibilities. More importantly, a delivery failure should be isolated from the ingestion path, while high webhook traffic may require the ingestion and delivery services to scale independently.

Component identification from Lab 2

Webhook Ingestion Service Ingest Inbound Webhook

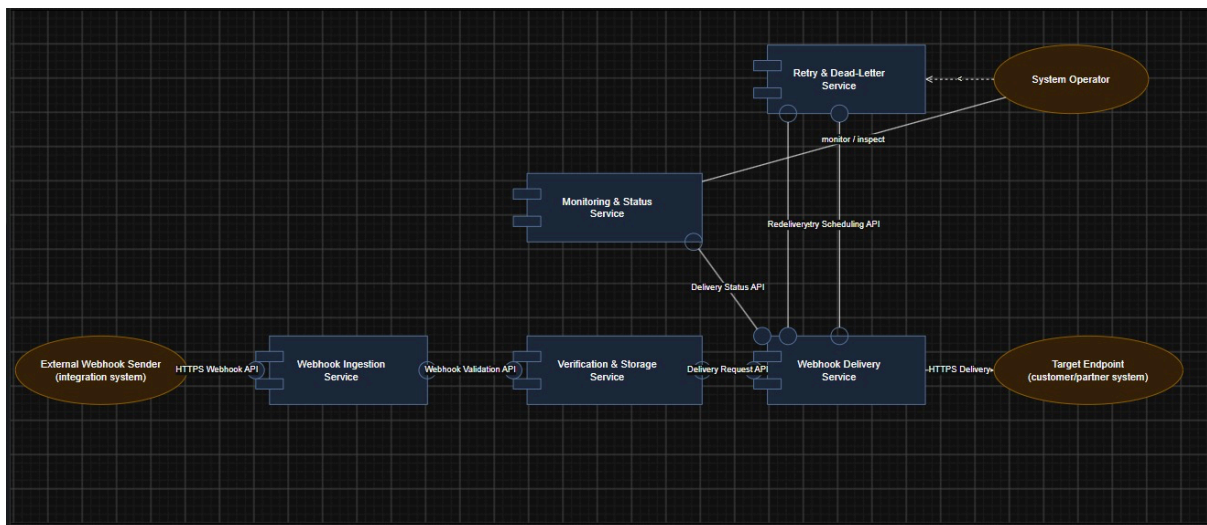
Verification & Storage Service Verify and Store Webhook

Webhook Delivery Service Deliver Webhook Payload

Retry & Dead-Letter Service Automatically Retry Failed Deliveries; Manually Retry Dead-Lettered Webhook

Monitoring & Status Service View Webhook Delivery Status

3. UML Component Diagram



4. Architectural Justification

Architecture Selection

We chose Microservices Architecture for the Webhook Ingestion & Retry Mechanism Hub because its main responsibilities can be separated into focused services with clear interfaces.

Reason 1 — Fault isolation

Webhook delivery can fail when a target endpoint is unavailable or returns an error. Separating delivery from ingestion means that a delivery problem can be handled by the retry path without stopping the rest of the system.

Reason 2 — Independent scaling

Webhook traffic can vary considerably. Ingestion and delivery may need more resources

than monitoring, so individual services can be scaled according to their own workload.

Security advantage

The Verification & Storage Service provides a clear boundary for validating incoming requests before they are stored or forwarded. This helps prevent malformed or unauthorised webhook requests from entering the delivery flow.

Performance benefit

The services can process different parts of the workflow independently. Ingestion, delivery, retry handling, and monitoring can therefore proceed concurrently, helping the system maintain throughput as event volume grows.

Component responsibilities

Webhook Ingestion Service

Accepts incoming webhook requests.

Verification & Storage Service

Validates and stores accepted webhook events.

Webhook Delivery Service

Sends events to the intended target endpoint.

Retry & Dead-Letter Service

Schedules retries and handles repeatedly failed events.

Monitoring & Status Service

Tracks delivery outcomes and supports operator inspection and manual retry.